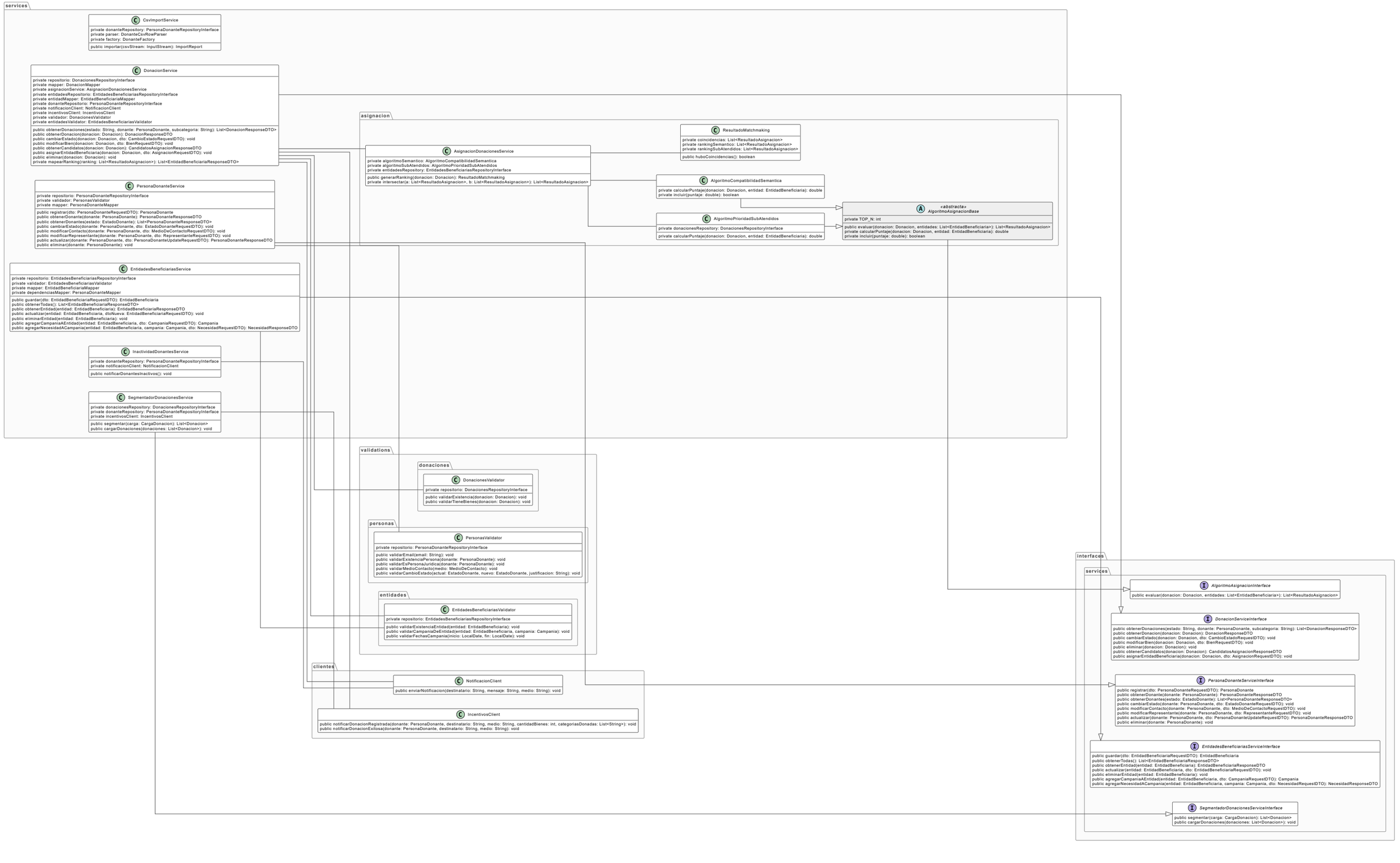
services
CsvImportService
private donanteRepository: PersonaDonanteRepositoryInterface
private parser: DonanteCsvRowParser
private factory: DonanteFactory
public importar(csvStream: InputStream): ImportReport
DonacionService
private repositorio: DonacionesRepositoryInterface
private mapper: DonacionMapper
private asignacionService: AsignacionDonacionesService
private entidadesRepositorio: EntidadesBeneficiariasRepositoryInterface
private entidadMapper: EntidadBeneficiariaMapper
private donanteRepositorio: PersonaDonanteRepositoryInterface
private notificacionClient: NotificacionClient
private incentivosClient: IncentivosClient
private validador: DonacionesValidator
private entidadesValidator: EntidadesBeneficiariasValidator
public obtenerDonaciones(estado: String, donante: PersonaDonante, subcategoria: String): List<DonacionResponseDTO>
public obtenerDonacion(donacion: Donacion): DonacionResponseDTO
public cambiarEstado(donacion: Donacion, dto: CambioEstadoRequestDTO): void
public modificarBien(donacion: Donacion, dto: BienRequestDTO): void
public obtenerCandidatos(donacion: Donacion): CandidatosAsignacionResponseDTO
public asignarEntidadBeneficiaria(donacion: Donacion, dto: AsignacionRequestDTO): void
public eliminar(donacion: Donacion): void
private mapearRanking(ranking: List<ResultadoAsignacion>): List<EntidadBeneficiariaResponseDTO>
PersonaDonanteService
private repositorio: PersonaDonanteRepositoryInterface
private validador: PersonasValidator
private mapper: PersonaDonanteMapper
public registrar(dto: PersonaDonanteRequestDTO): PersonaDonante
public obtenerDonante(donante: PersonaDonante): PersonaDonanteResponseDTO
public obtenerDonantes(estado: EstadoDonante): List<PersonaDonanteResponseDTO>
public cambiarEstado(donante: PersonaDonante, dto: EstadoDonanteRequestDTO): void
public modificarContacto(donante: PersonaDonante, dto: MedioDeContactoRequestDTO): void
public modificarRepresentante(donante: PersonaDonante, dto: RepresentanteRequestDTO): void
public actualizar(donante: PersonaDonante, dto: PersonaDonanteUpdateRequestDTO): PersonaDonanteResponseDTO
public eliminar(donante: PersonaDonante): void
EntidadesBeneficiariasService
private repositorio: EntidadesBeneficiariasRepositoryInterface
private validador: EntidadesBeneficiariasValidator
private mapper: EntidadBeneficiariaMapper
private dependenciasMapper: PersonaDonanteMapper
public guardar(dto: EntidadBeneficiariaRequestDTO): EntidadBeneficiaria
public obtenerTodas(): List<EntidadBeneficiariaResponseDTO>
public obtenerEntidad(entidad: EntidadBeneficiaria): EntidadBeneficiariaResponseDTO
public actualizar(entidad: EntidadBeneficiaria, dtoNueva: EntidadBeneficiariaRequestDTO): void
public eliminarEntidad(entidad: EntidadBeneficiaria): void
public agregarCampaniaAEntidad(entidad: EntidadBeneficiaria, dto: CampaniaRequestDTO): Campania
public agregarNecesidadACampania(entidad: EntidadBeneficiaria, campania: Campania, dto: NecesidadRequestDTO): NecesidadResponseDTO
InactividadDonantesService
private donanteRepository: PersonaDonanteRepositoryInterface
private notificacionClient: NotificacionClient
public notificarDonantesInactivos(): void
SegmentadorDonacionesService
private donacionesRepository: DonacionesRepositoryInterface
private donanteRepository: PersonaDonanteRepositoryInterface
private incentivosClient: IncentivosClient
public segmentar(carga: CargaDonacion): List<Donacion>
public cargarDonaciones(donaciones: List<Donacion>): void
asignacion
AsignacionDonacionesService
private algoritmoSemantico: AlgoritmoCompatibilidadSemantica
private algoritmoSubAtendidos: AlgoritmoPrioridadSubAtendidos
private entidadesRepository: EntidadesBeneficiariasRepositoryInterface
public generarRanking(donacion: Donacion): ResultadoMatchmaking
private intersectar(a: List<ResultadoAsignacion>, b: List<ResultadoAsignacion>): List<ResultadoAsignacion>
ResultadoMatchmaking
private coincidencias: List<ResultadoAsignacion>
private rankingSemantico: List<ResultadoAsignacion>
private rankingSubAtendidos: List<ResultadoAsignacion>
public huboCoincidencias(): boolean
AlgoritmoCompatibilidadSemantica
private calcularPuntaje(donacion: Donacion, entidad: EntidadBeneficiaria): double
private incluir(puntaje: double): boolean
AlgoritmoPrioridadSubAtendidos
private donacionesRepository: DonacionesRepositoryInterface
private calcularPuntaje(donacion: Donacion, entidad: EntidadBeneficiaria): double
«abstracta»
AlgoritmoAsignacionBase
private TOP_N: int
public evaluar(donacion: Donacion, entidades: List<EntidadBeneficiaria>): List<ResultadoAsignacion>
private calcularPuntaje(donacion: Donacion, entidad: EntidadBeneficiaria): double
private incluir(puntaje: double): boolean
validations
donaciones
DonacionesValidator
private repositorio: DonacionesRepositoryInterface
public validarExistencia(donacion: Donacion): void
public validarTieneBienes(donacion: Donacion): void
personas
PersonasValidator
private repositorio: PersonaDonanteRepositoryInterface
public validarEmail(email: String): void
public validarExistenciaPersona(donante: PersonaDonante): void
public validarEsPersonaJuridica(donante: PersonaDonante): void
public validarMedioContacto(medio: MedioDeContacto): void
public validarCambioEstado(actual: EstadoDonante, nuevo: EstadoDonante, justificacion: String): void
entidades
EntidadesBeneficiariasValidator
private repositorio: EntidadesBeneficiariasRepositoryInterface
public validarExistenciaEntidad(entidad: EntidadBeneficiaria): void
public validarCampaniaDeEntidad(entidad: EntidadBeneficiaria, campania: Campania): void
public validarFechasCampania(inicio: LocalDate, fin: LocalDate): void
clientes
NotificacionClient
public enviarNotificacion(destinatario: String, mensaje: String, medio: String): void
IncentivosClient
public notificarDonacionRegistrada(donante: PersonaDonante, destinatario: String, medio: String, cantidadBienes: int, categoriasDonadas: List<String>): void
public notificarDonacionExitosa(donante: PersonaDonante, destinatario: String, medio: String): void
interfaces
services
AlgoritmoAsignacionInterface
public evaluar(donacion: Donacion, entidades: List<EntidadBeneficiaria>): List<ResultadoAsignacion>
DonacionServiceInterface
public obtenerDonaciones(estado: String, donante: PersonaDonante, subcategoria: String): List<DonacionResponseDTO>
public obtenerDonacion(donacion: Donacion): DonacionResponseDTO
public cambiarEstado(donacion: Donacion, dto: CambioEstadoRequestDTO): void
public modificarBien(donacion: Donacion, dto: BienRequestDTO): void
public eliminar(donacion: Donacion): void
public obtenerCandidatos(donacion: Donacion): CandidatosAsignacionResponseDTO
public asignarEntidadBeneficiaria(donacion: Donacion, dto: AsignacionRequestDTO): void
PersonaDonanteServiceInterface
public registrar(dto: PersonaDonanteRequestDTO): PersonaDonante
public obtenerDonante(donante: PersonaDonante): PersonaDonanteResponseDTO
public obtenerDonantes(estado: EstadoDonante): List<PersonaDonanteResponseDTO>
public cambiarEstado(donante: PersonaDonante, dto: EstadoDonanteRequestDTO): void
public modificarContacto(donante: PersonaDonante, dto: MedioDeContactoRequestDTO): void
public modificarRepresentante(donante: PersonaDonante, dto: RepresentanteRequestDTO): void
public actualizar(donante: PersonaDonante, dto: PersonaDonanteUpdateRequestDTO): PersonaDonanteResponseDTO
public eliminar(donante: PersonaDonante): void
EntidadesBeneficiariasServiceInterface
public guardar(dto: EntidadBeneficiariaRequestDTO): EntidadBeneficiaria
public obtenerTodas(): List<EntidadBeneficiariaResponseDTO>
public obtenerEntidad(entidad: EntidadBeneficiaria): EntidadBeneficiariaResponseDTO
public actualizar(entidad: EntidadBeneficiaria, dto: EntidadBeneficiariaRequestDTO): void
public eliminarEntidad(entidad: EntidadBeneficiaria): void
public agregarCampaniaAEntidad(entidad: EntidadBeneficiaria, dto: CampaniaRequestDTO): Campania
public agregarNecesidadACampania(entidad: EntidadBeneficiaria, campania: Campania, dto: NecesidadRequestDTO): NecesidadResponseDTO
SegmentadorDonacionesServiceInterface
public segmentar(carga: CargaDonacion): List<Donacion>
public cargarDonaciones(donaciones: List<Donacion>): void